

Runtime parameter adaptation for a flow-queueing active queue discipline: a controlled evaluation on stock Linux

Amritha S

Yugeshwaran P

Deepti Annuncia

Vellore Institute of Technology, Chennai

Abstract

The default Linux queue discipline, `fq_codel`, exposes four parameters fixed at configuration time. Runtime adaptation of such parameters has been proposed repeatedly, but never evaluated under conditions that separate a controller’s computational cost from the effect of its decisions. We implement a controller that adapts all four parameters using an additive-increase multiplicative-decrease policy informed by eBPF flow telemetry, and evaluate it on an unmodified kernel against eight alternative queue disciplines under an identical bottleneck, with three repetitions per condition and a sham-controller condition that applies no change. Flow queueing with a delay target reduces 95th-percentile latency by 99.0 per cent relative to an unmanaged queue at indistinguishable goodput. Adapting that discipline’s parameters reduces mean queue occupancy by 9.6 per cent under a steady load and produces no improvement under a load whose flow count varies, while an existing queue discipline requiring no controller outperforms the adapted system by a larger margin than the adaptation achieves.

Introduction

Bufferbloat, persistently full network buffers that inflate queueing delay without affecting throughput, was identified by Gettys and Nichols over a decade ago[1] and remains the dominant source of latency on access links. The Linux response has converged on `fq_codel` [2], now the default queue discipline on most distributions, which combines Deficit Round Robin scheduling across roughly a thousand flow queues with an independent CoDel[3, 4] instance per queue.

`fq_codel` exposes four parameters that are fixed at configuration time and never change: `target` (5 ms), `interval` (100 ms), `limit` (10240 packets) and `quantum` (1514 bytes). Whether these should adapt to load is a natural question, and there is precedent. Adaptive RED[5] adapts RED’s maximum drop probability by an additive-increase multiplicative-decrease rule. Ye and Leung[6] derive stability conditions for CoDel and adapt its `interval` analytically. More recently, DESiRED[7] adapts the target delay of a P4 active queue management (AQM) implementation at runtime using deep reinforcement learning driven by in-band network telemetry, and AQM-LLM[8] distills a large language model into a controller for the Low Latency, Low Loss and Scalable throughput (L4S) architecture [9, 10]. A recent survey[11] catalogues the learning-based AQM literature and observes that most proposals are evaluated only in simulation.

Runtime adaptation of an AQM delay target is therefore established, and we do not claim it as novel. What is absent from this literature is a controlled evaluation of whether such adaptation helps. Every system cited above reports that its adaptation improved some quantity. None separates the computational cost of running a controller from the effect of its decisions; none reports the latency experienced by a sparse probe flow and by

the bulk flows separately, despite flow-queueing disciplines privileging sparse flows by design; none compares against the range of queue disciplines an operator could simply deploy instead; and none reports a null result.

We address that gap. We implement a userspace controller that adapts all four `fq_codel` parameters using the AIMD policy of Adaptive RED together with a gradient-based estimate of the congestion trajectory, instrumented by an extended Berkeley Packet Filter (eBPF) program at the Linux traffic control egress hook. We evaluate it on an unmodified Linux kernel against eight alternative queue disciplines under an identical bottleneck, with three independent repetitions per condition, two workloads, and a sham-controller condition that isolates computational cost from control decisions.

Our principal findings are that the presence of a flow-queueing discipline with a delay target reduces tail latency by 99.0% relative to an unmanaged first-in-first-out queue at indistinguishable goodput; that adapting that discipline’s parameters yields a 9.6% reduction in mean queue occupancy under a steady load and no improvement at all under a load whose flow count varies; and that CAKE[12], an existing queue discipline requiring no controller, outperforms the adapted system by a larger margin than the adaptation itself achieves.

Results

Every figure quoted below is generated from the run logs by `src/analyse.py`; none is typed by hand.

0.1 Flow queueing dominates

Table ?? gives all ten conditions. The largest effect by two orders of magnitude is the presence of a flow-

queueing discipline with a delay target, not the tuning of one. Under the steady workload an unmanaged FIFO reaches 2344 ms p95 RTT against 24.57 ms for `fq_codel` — a 99.0% reduction ($p < 0.001$) — at goodput that is statistically indistinguishable (9.453 vs 9.414 Mbps, $p = 0.20$). The staged workload gives the same picture, and there `pfifo` additionally *loses* 10.2% goodput and 1.4% fairness ($p = 0.015$, $p = 0.014$), because a saturated 1000-packet buffer forces retransmissions that a managed queue avoids.

Fair queueing without a delay target is not sufficient. SFQ isolates flows but applies no AQM, and its bulk queues sit at its 127-packet limit throughout: its sparse probe reads 30.68 ms while the bulk flows sharing the link read 294.8 ms. The AQM and the scheduler do distinct jobs and both are necessary.

0.2 Sparse and bulk latency are not interchangeable

Figure ?? plots the two measurements side by side. The ratio between them ranges from $1.1\times$ (CoDel, a single queue) to $9.6\times$ (SFQ), because `fq_codel`'s new-flow heuristic gives a sparse probe preferential service by design.

A single-probe measurement can therefore invert a ranking. By sparse probe alone SFQ (30.68 ms) appears to beat CoDel (39.02 ms); by the latency the bulk flows actually experience, SFQ is 294.8 ms against CoDel's 44.5 ms. We report both throughout, and we suggest any AQM evaluation should.

0.3 The controller's computational cost is negligible

The sham condition runs the controller at its full polling cadence and applies no parameter change. Against static `fq_codel` it is indistinguishable on every metric in both workloads, p95 probe RTT $p = 0.29$ (steady) and $p = 0.24$ (staged), backlog $p = 0.94$ and $p = 0.76$.

This matters for interpreting the next subsection. Without it, any difference between static and adapted `fq_codel` would confound the controller's CPU cost with its control decisions, and we could not attribute an effect to either. Having established the cost is nil, differences that remain are attributable to the decisions.

0.4 Adaptation: a small effect under steady load, none under change

Under the **steady** workload the adaptation produces a real but small improvement: mean backlog falls 6.79 from 6.14 packets, a 9.6% reduction ($p = 0.050$), and p95 probe RTT falls 0.5% ($p = 0.047$). Goodput and fairness are unchanged. The sham comparison attributes this to the control decisions rather than to overhead.

Under the **staged** workload, where the flow count changes twice during the run, the adaptation produces

no improvement on any metric. Mean probe RTT is 1.6% *worse* ($p = 0.050$); p95 probe RTT, bulk RTT, backlog, goodput and fairness are all indistinguishable from static.

That direction is worth stating plainly. The workload designed to give the controller something to respond to is the one in which it helps least. Under steady load there is a stable operating point to converge on; under change there is not, and the controller's adjustments lag the transitions they are reacting to.

0.5 An off-the-shelf alternative outperforms the adapted system

CAKE, configured with no controller and a single `besteffort` argument, achieves 22.9 ms p95 probe RTT under the steady workload against 24.43 ms for adapted `fq_codel` ($p < 0.001$), and 33.65 ms bulk RTT against 35.87 ms.

The margin by which CAKE beats the adapted system is larger than the margin by which the adaptation beats the static defaults. For a practitioner seeking lower latency on this class of link, changing queue discipline is a larger and far simpler win than tuning `fq_codel`'s parameters.

0.6 Predictive control never engaged

No adjustment in any run was tagged predictive. The cause is structural rather than a coding fault, and it survives the correction described in Section ??: an adjustment fires only once the regime has been unchanged for five consecutive samples, and the trajectory estimate is derived from the gradient over that same window, which by then is below the threshold that would classify it as **WORSENING** or **RECOVERING**. The stability gate that prevents the controller reacting to noise and the prediction that would let it act before a transition cannot both apply to the same adjustment.

We therefore withdraw the predictive-control contribution rather than claim it. Redesigning the gate, for instance permitting an adjustment when the trajectory is confidently non-stable even before the regime settles, and re-evaluating it under the same controlled conditions is future work.

0.7 Where adaptation does help: short paths

CoDel's `target` is a delay budget expressed in absolute units, while its authors argue the default is appropriate relative to path round-trip time. Sweeping the base RTT over 5, 20, 80 and 200 ms tests that directly.

Static `fq_codel` holds median queueing delay between 1.96 and 5.00 ms across the entire 40-fold span, bounded by its 5 ms target. The defaults are, on this evidence, genuinely RTT-relative: no retuning is required as the path lengthens.

Adaptation is nonetheless not uniformly inert. Measured on the RTT the bulk flows themselves experience:

Discussion

Base RTT	Static	Adapted	Change	Interpretation
5 ms	27.68 ± 0.14	22.58 ± 0.15	-18.4%	The dominant determinant of queueing latency on this class of link is the presence of flow queueing combined with a delay target, not the values of that mechanism's parameters. The gap between an unmanaged queue and any flow-queueing discipline is two orders of magnitude; the gap between default and adapted parameters is below one percent on latency. This is consistent with the position of CoDel's authors[3] that its defaults are expressed relative to path round-trip time by construction and require no tuning.
20 ms	36.39 ± 0.08	35.87 ± 0.80	-1.4%	
80 ms	89.30 ± 0.37	89.86 ± 0.70	$+0.6\%$	
200 ms	208.00 ± 0.76	207.47 ± 0.74	-0.3%	

* $p < 0.0001$; the remaining three are not distinguishable ($p = 0.38, 0.31, 0.43$).

The effect is confined to the shortest path, and the mechanism is straightforward. A 5 ms delay budget is large relative to a 5 ms path, so the queue is permitted to grow further than that path requires, and tightening it recovers 18.4% of the bulk flows' round-trip time. Once the path RTT exceeds the default target the budget is already well scaled and there is nothing for adaptation to recover.

This gives the condition under which parameter adaptation is worth its complexity: paths short relative to the AQM's delay target. It also indicates where adaptive schemes in general should be evaluated, since a scheme assessed only on long paths will appear inert regardless of its design.

0.8 Sparse flows under a mixed workload

Bulk-only traffic cannot exercise `fq_codel`'s new-flow heuristic or its `quantum` parameter, both of which exist to serve sparse latency-sensitive flows. Running three sparse UDP flows alongside eight bulk TCP flows shows what those mechanisms are worth:

System	Sparse loss (%)	Sparse jitter (ms)
pfifo, no AQM	7.97	2.19
FQ-PIE	0.000	1.91
CAKE	0.007	1.36
fq_codel	0.002	2.00

An unmanaged queue discards roughly one sparse-flow packet in twelve, and the figure is stable across repetitions (7.47%, 7.98%, 8.31%). Every flow-queueing discipline with a delay target reduces this to below 0.03%. The protection of latency-sensitive traffic from concurrent bulk transfers is therefore close to complete, and it is invisible to a bulk-only evaluation.

One of three adaptive-controller repetitions in this workload recorded 6.31% sparse loss against 0.000% and 0.029% in the others. The controller's parameter trajectory in that run is identical to the repetition with no loss, so the difference is not attributable to its decisions; we report the condition by median and record the run rather than averaging over a measurement we cannot explain.

Measurement practices

Three aspects of our methodology produced results that a conventional evaluation would have missed, and we suggest them as general practice.

First, sparse-probe and bulk-flow latency must be reported separately. Because `fq_codel` and related disciplines give a newly arriving sparse flow preferential service, an ICMP probe measures a privileged path. We observe ratios between probe and bulk latency ranging from 1.1 to 9.6 across disciplines, and the ranking of two disciplines inverts depending on which is quoted. Recent work by Ray et al.[13] documents a related effect in which the presence of an AQM changes what standard throughput measurements report.

Second, goodput must be computed from bytes received rather than bytes sent. Under a large unmanaged buffer the sender enqueues faster than the link drains, and the sender-side figure exceeded the link capacity in our measurements.

Third, a control system's computational cost must be separated from its control decisions. Our sham condition runs the controller at full polling cadence while applying no parameter change; it is statistically indistinguishable from the static configuration on every metric, which licenses attributing the observed differences to the controller's decisions rather than its overhead. Without this condition the two explanations are confounded.

Limitations

Our measurements are confined to a 10 Mbit bottleneck with 8 to 24 bulk TCP CUBIC[14] flows and a 20 ms base round-trip time, representative of a residential or access link rather than a datacentre or backbone.

All flows are bulk transfers; mixed workloads containing sparse latency-sensitive flows are precisely where `fq_codel`'s new-flow heuristic and an adaptive `quantum` might plausibly matter, and we have not tested them. Our null result should not be extrapolated to that setting. We evaluate a single bottleneck; multi-hop topologies with several congested links are not covered. Experiments run under hardware-assisted emulation, and we validated shaper accuracy under emulation before collecting results, but measurements at substantially higher link rates would require revalidation.

The predictive component of our controller never engaged. An adjustment fires only after the congestion regime has been unchanged for five consecutive samples, while the trajectory estimate is derived from the gradient over that same window and is therefore below its threshold by then. The stability gate that prevents reaction to noise and the prediction that would permit action before a transition cannot both apply to a single adjustment. We report this component as withdrawn rather than claim it; redesigning the gate and re-evaluating under the same controlled conditions is future work.

Outlook

Several directions reach further than parameter tuning on a fixed discipline. L4S[9, 10] attains sub-millisecond queueing delay by altering the congestion signalling contract between network and transport rather than by tuning a queue. Programmable data planes permit the queueing policy itself to be replaced[7], and proposals for a fully programmable eBPF queue discipline[15] would allow the same within Linux. Scheduling advances such as self-clocked round robin[16] address the round-robin scheduler underlying `fq_codel` rather than its AQM. For an operator seeking lower latency today, our measurements suggest that selecting an appropriate queue discipline is a larger and simpler improvement than tuning the parameters of one.

Methods

Topology

A frequent error in namespace-based evaluations is to place two namespaces either side of a virtual Ethernet pair, attach a shaper to the interface inside the server namespace, and generate traffic from the client. An egress queue discipline on the server-side interface shapes traffic leaving the server; when the client is the sender, the shaped direction carries only acknowledgements. Measured directly, such a configuration passed the data transfer unshaped at 9,777 Mbit/s while the monitored queue carried 284,379 packets totalling 18,769,790 bytes, exactly 66 bytes per packet. Every queue statistic collected on such a topology describes acknowledgement queueing.

We therefore use three network namespaces, representing a client, a router and a server, connected by two virtual Ethernet pairs. The shaper and the queue

discipline under test are attached to the router's egress interface toward the server, so that the bottleneck lies on the data path. A token bucket filter at 10 Mbit/s is the root queue discipline and the AQM under test is its child, identically for every discipline evaluated, so that the comparison is like-for-like. Network emulation adds 10 ms of delay at each edge, giving a 20 ms base round-trip time.

Queue disciplines evaluated

Nine configurations were measured: `pfifo` (no AQM), stochastic fair queueing, adaptive RED[17, 5], CoDel[4], PIE[18], FQ-PIE[19], CAKE[12], static `fq_codel`[2], and `fq_codel` under our controller. CAKE was configured in best-effort mode without its own bandwidth parameter so that the token bucket filter remained the sole shaper.

Controller

The controller operates on three timescales. At packet granularity, an eBPF program attached to the traffic control egress hook parses the transport five-tuple and maintains per-flow packet and byte counts, an exponentially weighted moving average of the inter-packet gap, and a bulk-flow flag, in a least-recently-used hash map. Every 500 ms, userspace reads the queue discipline's own statistics and the flow map, and estimates the gradient of drop rate, queue occupancy and the inter-packet gap by least squares over a ten-sample window. Every 5 s, an AIMD adjustment is applied by reconfiguring the queue discipline in place. The separation of the measurement and update timescales follows the two-timescale stochastic approximation framework[20].

Flow-map contents are read through the `bpf(2)` system call rather than by invoking an external utility per sample. Besides being approximately four times faster, this avoids introducing an additional process whose scheduling would confound the latency comparison that the sham condition exists to isolate.

Measurements

For each run we record: per-flow goodput and retransmission counts from the traffic generator, computed from bytes received; a 500 ms time series of the AQM's own queue statistics, parsed from the specific queue discipline rather than from whichever appears first in the kernel's output; a 20 Hz ICMP probe through the bottleneck, yielding approximately 1,200 samples per run; and the kernel's smoothed round-trip time estimate for every bulk connection at each reporting interval, yielding approximately 480 samples per run. Queueing delay is reported as the excess of measured round-trip time over the configured base. Fairness is Jain's index[21] over per-flow rates.

Experimental design

Two workloads were used. The steady workload maintains a constant eight flows. The staged workload varies the flow count during the run, from two to twenty-four and back, so that the congestion regime changes and the adaptive logic is exercised. Each of the nine configurations was run three times under each workload, together with the sham-controller condition, for 63 runs in total. Repetitions are independent runs rather than seeded variants, since the traffic generator exposes no seed; reported confidence intervals therefore describe run-to-run variance.

Differences between conditions are assessed with Welch's t -test, which does not assume equal group variances. Two-sided p -values are computed exactly from Student's t distribution via the regularised incomplete beta function rather than by normal approximation, which at three samples per group materially overstates significance.

Platform

Experiments were conducted on Linux 6.12.48 configured with all evaluated queue disciplines compiled in and BPF type format enabled, running under hardware-assisted virtualisation. The transport protocol is TCP CUBIC[14] throughout.

References

- [1] J. Gettys and K. Nichols. Bufferbloat: dark buffers in the Internet. *Commun. ACM*, 55(1):57–65, 2012.
- [2] T. Høiland-Jørgensen, P. McKenney, D. Täht, J. Gettys, and E. Dumazet. The FlowQueue-CoDel packet scheduler and active queue management algorithm. Technical Report RFC 8290, IETF, 2018.
- [3] K. Nichols and V. Jacobson. Controlling queue delay. *ACM Queue*, 10(5):20–34, 2012.
- [4] K. Nichols, V. Jacobson, A. McGregor, and J. Iyengar. Controlled delay active queue management. Technical Report RFC 8289, IETF, 2018.
- [5] S. Floyd, R. Gummadi, and S. Shenker. Adaptive RED: an algorithm for increasing the robustness of RED's active queue management. Technical report, ICSI, 2001.
- [6] J. Ye and K.-C. Leung. Adaptive and stable delay control for combating bufferbloat: theory and algorithms. *IEEE Syst. J.*, 14(1):1285–1296, 2020.
- [7] F. Rodriguez et al. DESiRED: dynamic, enhanced and smart iRED, a P4-AQM with deep reinforcement learning and in-band network telemetry. *Comput. Netw.*, 244, 2024.
- [8] D. Satish et al. Distilling large language models for network active queue management. *IEEE Trans. Netw.*, 2026.
- [9] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White. Low latency, low loss and scalable throughput (L4S) Internet service: architecture. Technical Report RFC 9330, IETF, 2023.
- [10] K. De Schepper, B. Briscoe, and G. White. Dual-queue coupled active queue management for L4S. Technical Report RFC 9332, IETF, 2023.
- [11] M. P. Toopchinezhad and M. Ahmadi. Machine learning approaches for active queue management: a survey, taxonomy and future directions. *Comput. Netw.*, 262, 2025.
- [12] T. Høiland-Jørgensen, D. Täht, and J. Morton. Piece of CAKE: a comprehensive queue management solution for home gateways. In *Proc. IEEE LANMAN*, 2018.
- [13] S. Ray, T. Sharma, J. Marques, P. Schmitt, F. Bronzino, and N. Feamster. Characterizing the impact of active queue management on speed test measurements. *arXiv:2511.19213*, 2025.
- [14] S. Ha, I. Rhee, and L. Xu. CUBIC: a new TCP-friendly high-speed TCP variant. *ACM SIGOPS Oper. Syst. Rev.*, 42(5):64–74, 2008.
- [15] C. Hung and P. Wang. eBPF qdisc: a generic building block for traffic control. In *Netdev 0x17*, 2023.
- [16] E. Sharafzadeh, R. Matson, J. Tourrilhes, P. Sharma, and S. Ghorbani. Self-clocked round-robin packet scheduling. In *Proc. USENIX NSDI*, 2025.
- [17] S. Floyd and V. Jacobson. Random early detection gateways for congestion avoidance. *IEEE/ACM Trans. Netw.*, 1(4):397–413, 1993.
- [18] R. Pan, P. Natarajan, F. Baker, and G. White. Proportional integral controller enhanced (PIE). Technical Report RFC 8033, IETF, 2017.
- [19] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil, and M. P. Tahiliani. FQ-PIE queue discipline in the Linux kernel: design, implementation and challenges. In *Proc. IEEE LCN*, 2019.
- [20] V. S. Borkar. Stochastic approximation with two time scales. *Syst. Control Lett.*, 29(5):291–294, 1997.
- [21] R. Jain, D.-M. Chiu, and W. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical Report TR-301, DEC, 1984.